API SECURITY ASSESSMENT

Burp Suite & OWASP crAPI

Security Assessment Report — Executive Portfolio Edition

5 Confirmed Findings | 12 Evidence Captures | 4 OWASP API Categories

Prepared by	Vardhan
Document ID	ASMT-CRAPI-2026-01
Version	2.1 — Professional Executive Edition
Assessment Type	Educational / Authorized Lab Assessment
Target	Local OWASP crAPI deployment (Docker)
Assessment Date	19 August 2026
Classification	Sanitized Portfolio Copy
Tools Used	Burp Suite Community, Docker, MailHog, Python Requests, PowerShell

Controlled Scope

All testing in this report was performed against a locally deployed, intentionally vulnerable OWASP crAPI lab environment. No unauthorized third-party systems, production environments, or real customer data were assessed. This document is a sanitized portfolio edition prepared for educational and professional demonstration purposes.

Document Purpose

This executive portfolio edition presents the findings of a hands-on API security assessment in a format suitable for technical hiring managers, security leaders, and peer review. Emphasis is placed on demonstrated impact, evidence discipline, and actionable remediation rather than theoretical risk scoring.

1. Document Control & Contents

Why this edition is different

This edition adds an executive risk dashboard, formal rules of engagement, positive control observations, multi-stage attack-chain analysis, a prioritized remediation roadmap with ownership mapping, evidence indexing, and clearer management-level framing while preserving the original hands-on evidence and technical rigor. Version 2.1 further improves visual alignment, typography consistency, and professional presentation standards.

Document Control

Field	Value	Purpose
Owner	Vardhan	Assessment author / tester
Scope	Local OWASP crAPI	Authorized educational lab only
Evidence	Sanitized screenshots	JWTs, cookies and passwords redacted
Scoring	Qualitative severity	No invented CVSS scores
Status	Complete	Technical testing and evidence collection complete
Distribution	Portfolio / Educational	Not for production risk decisions

Table of Contents

#	Section	Page
1	Document Control & Contents	2
2	Executive Summary & Risk Dashboard	3
3	Scope, Rules of Engagement & Architecture	4
4	Methodology, Positive Controls & Limitations	5
5	Vulnerability Portfolio Summary	6
6	F-01 — BOLA / IDOR: Mechanic Report	7–8
7	F-02 — Excessive Data Exposure / BOPLA	9
8	F-03 — BOLA / IDOR: Vehicle Location	10
9	F-04 — Broken Authentication: OTP Reset	11
10	F-05 — SSRF via mechanic_api	12
11	Attack Chaining & Systemic Observations	13
12	Prioritized Remediation Roadmap	14
13	Evidence Index, Skills & Conclusion	15

2. Executive Summary & Risk Dashboard

I performed a hands-on API security assessment against a locally deployed OWASP crAPI environment to validate how authentication, authorization, data exposure, API versioning and server-side URL handling behave under controlled adversarial testing. Burp Suite Community Edition was the primary testing platform, supported by Docker, MailHog, PowerShell and a focused Python utility where manual tooling was impractical.

The assessment confirmed five vulnerabilities across four OWASP API Security Top 10 (2023) categories. Three findings were rated **High** because they demonstrated cross-user data access or account takeover. Two were rated **Medium** because the demonstrated impact was significant but did not, within the tested scope, prove deeper internal compromise. The report deliberately distinguishes observed impact from hypothetical worst-case impact.

5 Confirmed Findings 3 High / 2 Medium	**12** Evidence Captures Fully sanitized	**4** OWASP Categories API1, API2, API3, API7	**100%** Local Scope Authorized lab only

Executive Risk View

Risk Domain	Findings	Highest Severity	Observed Outcome
Authorization	2	High	Mechanic report + vehicle location disclosure
Authentication	1	High	OTP reset leading to account takeover
Data Exposure	1	Medium	Email + vehicle UUID + author metadata
Server-Side Requests	1	Medium	External URL fetched by backend

Management Takeaway

The strongest pattern is inconsistent authorization. Authentication is present and correctly rejects missing tokens, but authenticated users can still retrieve objects belonging to other accounts. A second systemic concern is inconsistent protection across API versions, which enabled password-reset abuse through the older v2 endpoint while the newer v3 endpoint showed stronger controls. Addressing these two control families removes the majority of demonstrated risk.

Positive Control Observed

Removing the Bearer token from the vehicle-list API produced HTTP 401 / InvalidToken. This confirms that authentication was enforced on that endpoint; the critical weakness was object-level authorization after successful authentication. Documenting positive controls demonstrates balanced testing rather than solely hunting for failures.

3. Scope, Rules of Engagement & Architecture

Authorized Scope

- Target: locally deployed OWASP crAPI running in Docker Desktop.
- Primary application endpoint: http://localhost:8888.
- Local MailHog endpoint: http://localhost:8025 (for OTP / email capture).
- Two dedicated crAPI test accounts were used for cross-user authorization validation.
- No production system, real customer data or unauthorized third-party target was assessed.

Rules of Engagement

- Testing was limited exclusively to the intentionally vulnerable lab environment.
- No destructive actions were performed beyond controlled password reset of a lab account.
- SSRF validation stopped after confirming external server-side fetching; internal metadata, loopback escalation and private network enumeration were intentionally not pursued.
- All evidence was sanitized before inclusion in this portfolio edition (JWTs, session cookies, passwords redacted).
- Findings are based strictly on observed behavior; hypothetical escalation is labeled as potential impact, not as demonstrated compromise.

Lab Architecture Snapshot

Burp Chromium	Burp Proxy	crAPI Web	API Services	Data & Support
User workflow	Capture / Replay	localhost:8888	identity / workshop / community	Postgres / Mongo / MailHog

Traffic flowed from the Burp Chromium browser through Burp Proxy to the crAPI web application and backend APIs. HTTP History served as the passive capture layer, while Repeater and Intruder were used only when a specific request required controlled manipulation.

Environment Troubleshooting

The assessment began with infrastructure troubleshooting. `docker ps` showed no running containers, while `docker ps -a` confirmed that the existing crAPI containers had stopped. After restarting the stack, later Vehicle Service History requests returned 502 Bad Gateway because `crapi-workshop` and `crapi-community` were not healthy. Restarting those services and waiting for healthy status restored HTTP 200 responses. This was treated as an environment issue, not a vulnerability.

Professional Practice

Documenting setup failures separately from security findings prevents operational issues from being misreported as vulnerabilities and demonstrates control over the test environment. This separation is a hallmark of mature assessment methodology.

4. Methodology, Positive Controls & Limitations

Assessment Workflow

crAPI Application → Burp Proxy → HTTP History → Endpoint Identification → Baseline Replay → Controlled Change → Response Comparison → Validation → Evidence Capture → Sanitization → Remediation Analysis

Intercept was kept OFF for most of the exercise so application workflows could complete normally while HTTP History captured the requests. For each security-sensitive endpoint, I first established a legitimate baseline and then changed one condition at a time — authenticated user, object identifier, API version or URL parameter — so the resulting behavior could be attributed to that change.

Testing Techniques

- Authenticated request capture and JWT / Bearer header analysis.
- Two-user object-level authorization testing (User 1 vs User 2).
- Request replay and parameter manipulation in Burp Repeater.
- Response-property review for unnecessary sensitive fields (BOPLA).
- API-version comparison during password-reset testing.
- Burp Intruder configuration for four-digit OTP testing.
- Controlled Python Requests + ThreadPoolExecutor automation when Community Intruder throughput became impractical.
- Server-side URL handling validation through the mechanic_api parameter.

Limitations & Boundaries

- This was a focused portfolio assessment, not an exhaustive code review or full endpoint inventory.
- No source-code analysis, formal CVSS scoring or production risk quantification was performed.
- The OTP automation was limited to the local lab and stopped immediately after the valid OTP was found.
- The SSRF test confirmed external fetching only; internal network and metadata-service escalation were intentionally not tested.
- Severity reflects demonstrated impact in the lab and is not presented as a production severity rating.

Evidence Standard

A finding was considered confirmed only when the request/response behavior or resulting application state visibly supported the conclusion. Scanner-only or speculative issues were not included. All screenshots were sanitized to remove authentication material before inclusion.

Primary Tools & Environment

Tool	Role	Notes
Burp Suite Community	Proxy, Repeater, Intruder, HTTP History	Primary interactive testing platform
Docker Desktop	crAPI stack orchestration	Containers restarted for healthy state
MailHog	Local SMTP capture	OTP email observation
Python 3 + Requests	OTP brute-force utility	ThreadPoolExecutor, early-stop on success
PowerShell	Supporting scripts / validation	Environment checks

5. Vulnerability Portfolio Summary

ID	Finding	OWASP	Severity	Priority	Status
F-01	BOLA / IDOR — Mechanic Report	API1	High	Immediate	Confirmed
F-02	Excessive Data Exposure / BOPLA	API3	Medium	Near-term	Confirmed
F-03	BOLA / IDOR — Vehicle Location	API1	High	Immediate	Confirmed
F-04	Broken Authentication — OTP Reset	API2	High	Immediate	Confirmed
F-05	Server-Side Request Forgery	API7	Medium	Near-term	Confirmed

Risk Matrix (Qualitative)

Impact / Likelihood	Rare	Unlikely	Possible	Likely	Almost Certain
Critical					
High			F-04	F-01 / F-03	
Moderate			F-05	F-02	
Low					
Minimal					

Severity Philosophy

High was reserved for demonstrated cross-user exposure or account takeover. Medium was used where significant behavior was proven but deeper escalation was not tested. This keeps the report credible and avoids inflating risk. No CVSS scores were invented; qualitative ratings are grounded solely in observed lab impact.

F-01	Broken Object Level Authorization — Mechanic Report	High	Immediate

OWASP: API1:2023 — Broken Object Level Authorization

Affected Endpoint: `GET /workshop/api/mechanic/mechanic_report?report_id=7`

Precondition: User 1 created a mechanic service request and legitimately owned report_id=7.

I first established the authorized baseline using User 1. The request returned HTTP 200 and exposed report 7, including the lab vehicle, owner information and service details. This baseline was necessary before changing the authenticated identity.

```
GET /workshop/api/mechanic/mechanic_report?report_id=7 HTTP/1.1
Host: localhost:8888
sec-ch-ua-platform: "Windows"
Authorization: Bearer <JWT REDACTED>

Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:8888/login
Accept-Encoding: gzip, deflate, br
Cookie: chat_session_id=<REDACTED>
```

Figure 1 — Authorized User 1 request for report_id=7. Authentication material is redacted.

Request Response

Pretty Raw Hex Render

```
Connection: keep-alive
Allow: GET, HEAD, OPTIONS
Vary: origin, Cookie
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin
Content-Length: 465

{
    "id":7,
    "mechanic":{
        "id":1,
        "mechanic_code":"TRAC_JHN",
        "user":{
            "email":"jhon@example.com",
            "number":""
        }
    },
    "vehicle":{
        "id":102,
        "vin":"R93LBLAKSX144TP9G",
        "owner":{
            "email":"vardhan_crapi@example.com",
            "number":"<REDACTED>"
        }
    },
    "problem_details":"Testing vehicle service request for API security assessment. The vehicle is showing an unusual engine warning and requires inspection.",
    "status":"pending",
    "created_on":"19 August, 2026, 02:11:06",
```

Figure 2 — Authorized baseline response for report 7 showing the lab owner and vehicle data.

Baseline Result: User 1 → report_id=7 → HTTP 200. This is expected behavior and serves as the control case.

F-01	Cross-User Validation — Mechanic Report BOLA	High	Immediate

OWASP: API1:2023 — Broken Object Level Authorization

To test object-level authorization, I repeated the same request using a second independent crAPI account. The resource identifier remained report_id=7, but the Bearer token belonged to User 2. A secure implementation should have rejected access because User 2 did not own the report.



Figure 3 — User 2 requests User 1's report_id=7. JWT and session data are redacted.



Figure 4 — The server still returns HTTP 200 and User 1's report, confirming BOLA.

Expected Behaviour: 403 Forbidden, 404 Not Found, or an equivalent ownership-based denial.

Observed Behaviour: HTTP 200 with the same User 1 report returned to User 2.

Security Impact: An authenticated attacker could access other customers' mechanic reports if report identifiers are discovered or enumerated. The exposed data included vehicle and owner information.

Recommended Remediation: Enforce object-level authorization on every mechanic-report lookup. Scope the database query to resources owned by or explicitly shared with the authenticated principal, rather than fetching by report_id first and checking only that a token exists.

Why this matters

Authentication answered who the caller was, but the API did not answer whether that caller was allowed to access this specific object. That distinction is the core of BOLA (Broken Object Level Authorization).

F-02	Excessive Data Exposure / Broken Object Property Level Authorization	Medium	Near-term

OWASP: API3:2023 — Broken Object Property Level Authorization

Affected Endpoint: `GET /community/api/v2/community/posts/recent?limit=30&offset=0`

Opening the Community page returned post content together with author properties that were not necessary for simply rendering a public post. The response exposed lab email addresses, vehicle UUIDs, author IDs and account-creation timestamps. The vehicle UUID became directly useful in the next authorization test.



Figure 5 — Community posts request captured in Burp Repeater. Authentication material is redacted.



Figure 6 — Response exposes Robot's email, vehicle UUID and additional author metadata.

Exposed Properties: nickname, email, vehicleid, authorid and account creation metadata.

Security Impact: The disclosure increases privacy exposure and gives an attacker stable object identifiers that can be reused against other endpoints. In this assessment, the leaked vehicle UUID enabled the vehicle-location BOLA test.

Recommended Remediation: Use response DTOs / serializers that return only fields required by the client. Treat internal identifiers, vehicle UUIDs and account metadata as sensitive by default, and review API responses independently of frontend visibility.

> **Attack-Enabling Data**
> The vehicle UUID was not just unnecessary data; it became an input to a higher-impact authorization vulnerability. Data minimization reduces both direct exposure and attack-chain opportunities.

F-03	BOLA / IDOR — Unauthorized Vehicle Location Disclosure	High	Immediate

OWASP: API1:2023 — Broken Object Level Authorization

Affected Endpoint: `GET /identity/api/v2/vehicle/4bae9968-ec7f-4de3-a3a0-ba1b2ab5e5e5/location`

The Community API exposed Robot's vehicle UUID. I supplied that UUID to the vehicle-location endpoint while authenticated as another test user. The server returned HTTP 200 with Robot's name, email and precise latitude/longitude instead of enforcing ownership.



Figure 7 — Authenticated request for another user's vehicle UUID. JWT and session data are redacted.

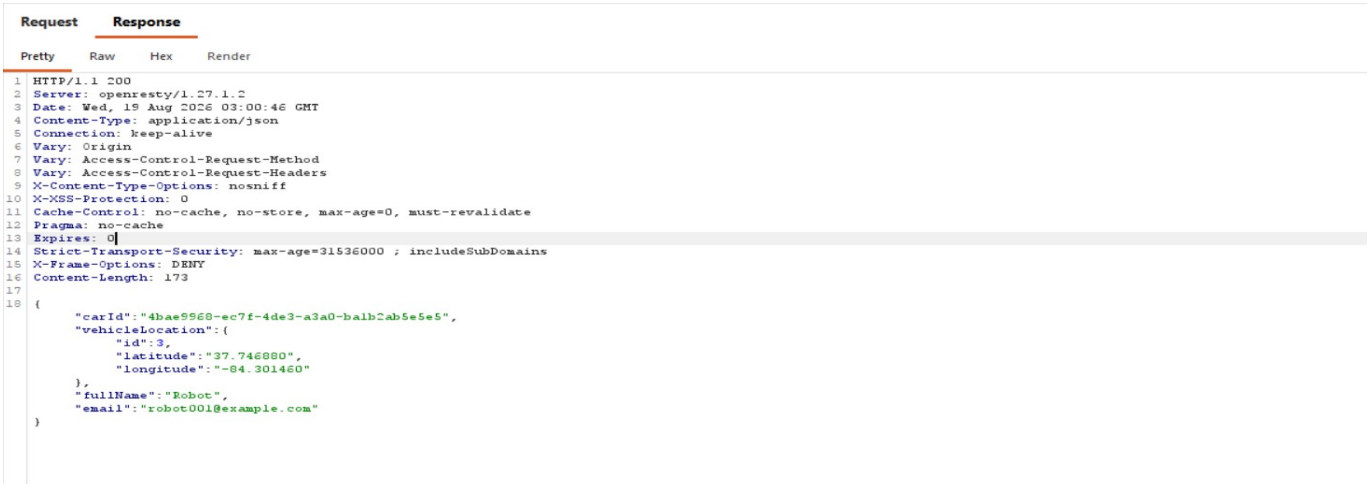


Figure 8 — HTTP 200 response exposes Robot's identity and precise vehicle coordinates.

Expected Behaviour: The caller should receive no location data for a vehicle they do not own or have explicit permission to view.

Observed Behaviour: HTTP 200 returned carId, latitude, longitude, fullName and email for another user.

Security Impact: This finding combines cross-user identity disclosure with precise location exposure. Even in a lab, location data is materially more sensitive than ordinary profile information because it can reveal physical movement or presence.

Recommended Remediation: Authorize the requested vehicle against the authenticated user before returning location data. Avoid resolving arbitrary vehicle UUIDs outside the caller's ownership scope, and apply consistent access checks across all vehicle endpoints.

F-04	Broken Authentication — OTP Brute Force Leading to Account Takeover	High	Immediate

OWASP: API2:2023 — Broken Authentication

Relevant Endpoints: `POST /identity/api/auth/forget-password`; `POST /identity/api/auth/v3/check-otp`; `POST /identity/api/auth/v2/check-otp`

The password-reset flow generated a four-digit OTP. The newer v3 validation endpoint showed stronger attempt handling, but the older v2 endpoint continued accepting guesses. I initially configured Burp Intruder, then switched to a small Python Requests utility because Community Edition throughput made testing the 0000–9999 space impractical. The utility used ThreadPoolExecutor and stopped immediately when a successful response was found.

```
PS C:\Users\vardh> notepad crapi_otp_test.py
PS C:\Users\vardh> py crapi_otp_test.py
[+] Starting fast OTP test...
[+] Target: http://localhost:8888/identity/api/auth/v2/check-otp
[*] Tested approximately 500/10000 OTPs
[*] Tested approximately 1000/10000 OTPs
[*] Tested approximately 1500/10000 OTPs
[*] Tested approximately 2000/10000 OTPs
[*] Tested approximately 2500/10000 OTPs
[*] Tested approximately 3000/10000 OTPs
[*] Tested approximately 3500/10000 OTPs
[*] Tested approximately 4000/10000 OTPs

==============================
[+] VALID OTP FOUND!
==============================
OTP: 4221
Status: 200
Response: {"message":"OTP verified","status":200}
New Password: <PASSWORD REDACTED>
```

Figure 9 — Controlled local automation found the valid OTP and received HTTP 200. The test password is redacted.

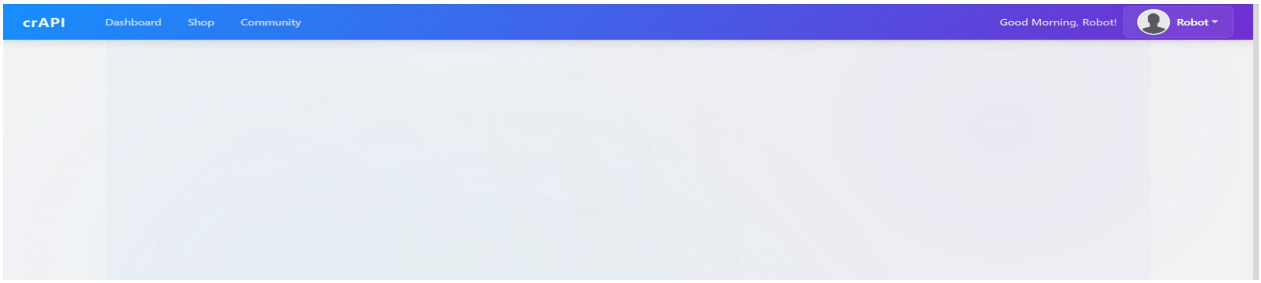


Figure 10 — Successful login to the Robot lab account after the reset, confirming account takeover.

Observed Behaviour: A valid OTP was discovered on the legacy v2 endpoint and the password reset completed. Login to robot001@example.com then succeeded with the controlled replacement password.

Security Impact: The demonstrated impact is account takeover when an attacker knows a target email and can reach the insufficiently protected legacy reset endpoint.

Recommended Remediation: Centralize password-reset protection so every API version shares the same attempt counter, rate limits and expiry state. Add per-account and IP/device throttling, progressive backoff, monitoring and short OTP lifetime. Remove or disable insecure legacy endpoints instead of leaving them reachable alongside hardened versions.

Key Engineering Lesson

Protecting only the newest API version is not enough. Legacy routes are part of the active attack surface until they are removed or enforce the same security controls.

F-05	Server-Side Request Forgery via mechanic_api	Medium	Near-term

OWASP: API7:2023 — Server-Side Request Forgery

Affected Endpoint: `POST /workshop/api/merchant/contact_mechanic`

Vulnerable Parameter: `mechanic_api`

The normal Contact Mechanic request supplied a backend callback URL in mechanic_api. I changed the destination to http://www.google.com. The first manual edit contained invalid JSON and correctly returned 400 Bad Request. After correcting the syntax, crAPI returned HTTP 200 and embedded the fetched Google HTML inside response_from_mechanic_api.



Figure 11 — Client-controlled mechanic_api is changed to http://www.google.com. Authentication data is redacted.

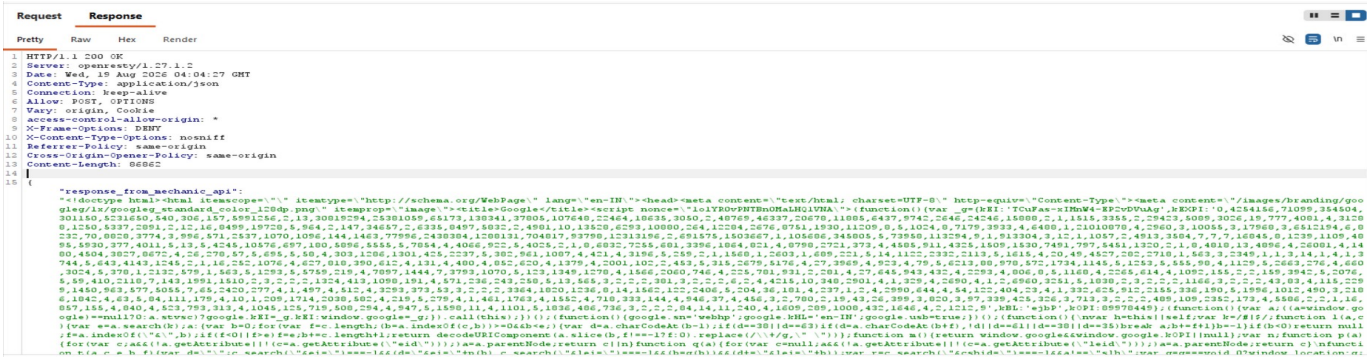


Figure 12 — HTTP 200 response contains response_from_mechanic_api with recognizable Google HTML.

Observed Behaviour: The backend performed a server-side request to a client-supplied external destination and returned the remote response.

Security Impact: External server-side fetching is confirmed. If the same control weakness also permits internal, loopback, link-local or cloud metadata destinations, impact could be higher; those escalation paths were intentionally not tested.

Recommended Remediation: Replace arbitrary client-supplied callback URLs with server-managed destinations or a strict allowlist. Validate resolved IPs and redirects, block private/loopback/link-local ranges, restrict protocols and apply outbound egress controls.

Evidence Discipline

The report rates this Medium because external fetching was demonstrated, while internal-network compromise was not. This avoids claiming a worst-case SSRF impact that was not actually tested.

11. Attack Chaining & Systemic Security Observations

Attack Chain A — Information Disclosure to Location Exposure

1. Community API	2. Identifier Reuse	3. Authorization Failure	4. Impact
Email + vehicle UUID exposed	UUID copied to vehicle endpoint	No ownership check enforced	Precise coordinates + identity returned

This chain shows why “excessive data exposure” should not be treated as a cosmetic privacy issue. Stable identifiers can become the bridge to a higher-impact authorization failure elsewhere in the API.

Attack Chain B — Legacy API to Account Takeover

1. Known Email	2. Reset Requested	3. v2 OTP Guessing	4. Valid OTP	5. Robot Login
Target email known	forget-password called	Legacy endpoint accepts guesses	OTP verified	Account takeover complete

The key weakness was not the existence of an OTP alone; it was inconsistent protection across API versions. The hardened route did not eliminate the risk while the older route remained reachable.

Systemic Themes

- Authentication without object-level authorization is insufficient.
- Response minimization is a security control, not just an API design preference.
- Legacy API versions must be treated as active production attack surface until removed.
- Client-controlled URLs require strict server-side destination policy and egress restrictions.
- Operational reliability and security validation are connected: unhealthy services can distort test results if not separated from actual findings.

Security Architecture View

The findings are not five unrelated bugs. They reveal recurring control gaps across authorization, data minimization, lifecycle management and outbound request policy. Addressing those control families will remove entire classes of similar defects.

12. Prioritized Remediation Roadmap

The following roadmap prioritizes the demonstrated risk rather than simply listing fixes by finding number. The goal is to remove the highest-impact attack paths first and then build controls that prevent recurrence.

Immediate — 0 to 7 Days

- Enforce ownership checks on mechanic-report and vehicle-location endpoints.
- Disable or remove the weak v2 OTP validation endpoint, or immediately apply the same rate-limiting state as v3.
- Restrict mechanic_api to approved server-managed destinations or a strict allowlist.
- Add alerting for repeated OTP failures and anomalous cross-object access.

Near-Term — Within 30 Days

- Centralize authorization middleware/policy checks so object ownership logic is not reimplemented inconsistently.
- Introduce response DTOs / schemas that explicitly whitelist client-visible properties.
- Inventory all active and deprecated API versions and remove unused routes.
- Add outbound egress restrictions and DNS/IP validation for any server-side HTTP client.
- Create automated regression tests for cross-user access denial and password-reset throttling.

Maturity Improvements — 60 to 90 Days

- Adopt API security test cases in CI/CD for BOLA, BOPLA, authentication abuse and SSRF.
- Maintain an API inventory with owners, versions, deprecation dates and data classifications.
- Implement structured security logging with object IDs, caller identity, authorization decisions and outbound destinations.
- Review sensitive data exposure periodically as frontend requirements change.
- Run recurring authorized assessments to verify that fixes remain effective across new versions.

Control Ownership

Control Area	Primary Owner	Evidence of Completion
Object Authorization	API / Backend Engineering	Cross-user tests return 403/404
Password Reset	Identity / IAM Engineering	Shared rate limits across all versions
Response Minimization	API Owners	Schemas exclude internal IDs and excess PII
SSRF / Egress	Backend + Platform / Network	Allowlist + egress policy + redirect validation
Monitoring	Security / SOC	Alerts for brute force, enumeration and unusual egress

13. Evidence Index, Skills & Conclusion

Evidence Index

Figure	Finding	Evidence	Sanitized
1–4	F-01	Authorized baseline + cross-user report access	Yes
5–6	F-02	Community request + leaked author properties	Yes
7–8	F-03	Foreign vehicle UUID request + location response	Yes
9–10	F-04	Valid OTP result + successful Robot login	Yes
11–12	F-05	Modified mechanic_api + fetched Google HTML	Yes

Skills Demonstrated

- Burp Proxy / HTTP History, Repeater and Intruder.
- JWT-based authentication analysis and two-user authorization testing.
- BOLA / IDOR validation using object IDs and UUIDs.
- Response-property analysis for BOPLA / excessive data exposure.
- API versioning and password-reset security analysis.
- Python Requests and ThreadPoolExecutor for narrowly scoped local automation.
- SSRF validation through a client-controlled callback URL.
- Docker troubleshooting and service health validation.
- Security evidence handling, sanitization, severity judgment and remediation writing.

Conclusion

The technical testing phase of this API Security Assessment was completed successfully against the controlled OWASP crAPI lab. Five vulnerabilities were confirmed with direct evidence and mapped to relevant OWASP API Security categories. The most significant outcomes were cross-user object access, precise vehicle-location disclosure and account takeover through an insufficiently protected legacy password-reset endpoint.

The assessment also demonstrated balanced testing: a missing JWT was correctly rejected with HTTP 401, infrastructure failures were separated from security findings, SSRF impact was not overstated beyond the external fetch that was actually proven, and every public evidence image was sanitized. That combination of technical validation, restraint and clear remediation is what makes the work representative of a professional security assessment rather than a collection of lab screenshots.

Assessment Status: COMPLETE

Hands-on testing, five confirmed findings, sanitized evidence collection and professional reporting are complete. The project is ready for portfolio publishing after final review.

Evidence Handling Note

JWT/Bearer tokens, chat_session_id values, the controlled test password and an unnecessary phone number are redacted. Lab emails, object identifiers and the consumed OTP are retained only where required to understand the demonstrated findings.